Michelle T. George

Advanced Problem-Solving Lab – High-IQ Track

Activity H1 – Solve the "Hidden Problem" Behind the Stated Problem

Title: From Surface Problem to Real Problem

Goal

Train yourself to see the *real* problem, not just the stated one.

Instructions

You are given a "client statement":

"Our exam system is too slow. We want a Python script to speed it up."

1. Identify at least 3 possible *real* problems this statement might hide.
   - Hidden problem 1: Inefficient Database queries
   - Hidden problem 2: Network latency or large amount
   - Hidden problem 3: Concurrency bottlenecks
2. For each hidden problem, state:
   - What data would you need to confirm it?
     - Problem 1: query execution time, slow query logs, database CPU usage.
     - Problem 2: Network response time, server loading time.
     - Problem 3: Number of concurrent users, server request handling time
3. Choose ONE hidden problem and re-frame it as a clear technical problem statement (2–3 lines):

New problem statement:

The exam system experiences delays due to inefficient database queries, causing slow times when retrieving student data. The system needs optimization in query execution to handle requests faster and improve overall performance

4. Design a very small Python experiment that could validate or reject your reframed problem (pseudo-code is fine):

python

 # Experiment idea (what will you measure, on what data, how?)

Activity H2 – Multiple Competing Solutions

Title: Design 3 Solutions Before Coding 1

Goal

Avoid fixation. Explore at least three distinct solution strategies.

Instructions

Problem:

"You receive a stream of log lines (text). You must detect 'suspicious' activity in real time."

1. Give three different solution ideas, each using a distinct main technique or data structure.
   - Approach A (e.g., rule-based / regex / thresholds): Regex for failed-login. flag if > 5/min
   - Approach B (e.g., statistical / sliding window / counters): count logs per IP in last 60s, with deque. flag spikes
   - Approach C (e.g., simple ML / anomaly detection concept): isolation forest on log features to find anomalies.
2. For each approach, list pros and cons (at least one each).
   - Approach A – Pros: fast Cons: misses new attack
   - Approach B – Pros: catches burst Cons: needs tuning
   - Approach C – Pros: finds unknown Cons: slow, needs data
3. Choose one approach to implement first and justify:

I choose approach ___ *because* it's fastest to build and test

4. Sketch high-level Python structure (modules/classes, not full code):

python

 # Modules / classes / key functions list

```
class Detector:
    def check(line):
    def mains():
```

Activity H3 – Self-Constrained Challenge: "Elegant Code Only"

Title: Solve with Elegance (Clarity, Minimalism, Correctness)

Goal

Produce code that is not only correct, but also minimal and beautiful.

Problem

Implement a mini "pipeline" for processing a list of tasks:

- Each task has an ID, description, and status ("todo", "doing", "done").
- You must:
  - Load tasks from a JSON file.
  - Filter by status.
  - Sort by ID.
  - Print a clean summary.

Constraints (you impose on yourself)

1. Maximum 25 lines of *real* Python (excluding blanks and comments).
2. No function longer than 7 lines.
3. Names must be meaningful and short.

Steps

1. Design your data model in 2–3 lines:

Task = dict with keys: id, desc, status

List of task: list [dict]

2. Decide at least one thing you will *not* do to keep it minimal:

No error handling or input validation Assume JSON is valid

3. Write your code here (respecting constraints):

```
import json, sys
status = sys.argv[1]
with open("task.json" as f:
task = json_load(f)
todo = [t for t in tasks if t["status"] = status]
todo.sort(key = lambda t: t["id"])
for t in todo:
print(f"{t['id']}: {t['desc']}")
```

python

 # Elegant pipeline solution

4. Reflection:
   - What did you have to sacrifice (if anything) to keep elegance?

error handling and CLI helps flags. kept only the core pipeline

Activity H4 – Complexity and Scaling Thought Experiment

Title: How Will Your Solution Break at Scale?

Goal
Develop realistic intuition about performance and scaling.

Instructions
Take ANY non-trivial function or algorithm you have recently written (search, summarization, scoring, etc.).

1. Describe what it does, in *one line*:
filters a JSON task line by status and sorts by id

2. Estimate its time complexity in Big-O notation, and justify in 1–2 lines:
   - Big-O: O(nlogn)
   - Because: filtering is O(n), sorting is O(nlogn). sort dominates

3. Scaling scenarios – imagine input sizes:
   - n = 10^3 (1,000)
   - n = 10^6 (1,000,000)
   - n = 10^9 (1,000,000,000)

For each, predict: will it still be usable (Y/N) and why?

n	Usable (Y/N)	Reason (1 short sentence)
10^3	Y	Runs in <1ms, instant
10^6	Y	Takes no. 5s, still fine for (2)
10^9	N	uses ~16 GB RAM and 30s + to sort, too slow

4. Suggest one change to improve scalability and briefly describe how it helps:
load tasks from a database with where status = ? ORDER By id index Helps by avoiding full file load and python sort - DB does it faster

Activity H5 – Adversarial Thinking: Break Your Own Design

Title: Attack Your Own Program

Goal

Develop robustness and defensive mindset.

Instructions

Choose one of your own projects or mini-tools (file reader, API client, CLI, etc.).

1. Write a short description of what it is supposed to do:
   Read tasks.Json, filters task by status arg, sort by id points
2. Now wear the hat of an *attacker* or malicious user.
   List 5 ways to misuse / break it:
   - Attack 1: delete tasks.json before running
   - Attack 2: give invalid status like xyz
   - Attack 3: Use 5GB json file to fill RAM
   - Attack 4: JSON with task missing id or disc keys
   - Attack 5: No command-line arg given
3. For each attack, suggest one defence in plain language:
   - Defence for attack 1: check if file exist, exit if not
   - Defence for attack 2: whitelist only 'todo'/'doing'/'done'
   - Defence for attack 3: limit file size to 10MB before loading
   - Defence for attack 4: use .get() with defaults for missing keys
   - Defence for attack 5: check len(sys.argv)>1 or show usage.
4. Implement at least one of these defences in your code and paste the updated snippet:

python

```
# Hardened code snippet:
import json, sys, os
if len(sys.argv)<2: exit("usage: app.py <status>")
status = sys.argv[1]
if status not in ["todo","doing","done"]: exit("Invalid status")
if not os.path.exists("tasks.json"): exit("no file")
with open("tasks.json") as F: tasks=json.load(F)
out = [t for t in tasks if t.get("status")==status]
out.sort(key=lambda t: t.get("id",0))
for t in out: print(f"{t.get('id',0)} : {t.get('desc','')}")
```

Activity H6 – Cross-Domain Transfer

Title: Apply Known Pattern to a New Domain

Goal

Practice lifting a known idea/pattern from one domain to a different one.

Instructions

Pick a pattern you already know well, e.g.:

- Sliding window
- Two-pointer technique
- Dynamic programming table
- Producer–consumer
- Map–reduce style
- Pipeline of filters

1. Name of pattern: Sliding Window
2. Original domain where you learned it (e.g., arrays, strings):

Strings and array - Used to find the longest substring without repeating character or max sum of k consecutive elements in a list

3. New domain to apply it in (e.g., logs, time series, learning activities, medical data):

Server logs / time series data - detecting traffic spikes or error bursts in minute logs.

4. Describe how you would adapt the pattern to this new domain, in 5–8 lines (can include pseudo-code):

In array we slide a fixed-size window to track sums. For server logs, the window is a time period like 5 min. Instead of array indexes, we use timestamps. We slide the window over log entries sorted by time. As we move forward, we add new log entries that enter the window & remove old ones that fall out. We track count of errors inside the window. If error count > threshold, we flag an alert for a traffic / error spike

5. Optional: implement a small prototype showing this transfer:

python

```
# Pattern transfer prototype

from collection import deque
from time import time
window, LIMIT = deque(), 60
def add_log(ts, msg):
    window.append((ts, msg))
    while window and ts - window[0][0] > 60:
        window.popleft()
    if len(window) > 50: print("alert: error spike")
```

Activity H7 – **Meta-Cognition: Your Personal Problem-Solving Manual**

Title: Write Your Own "How I Solve Problems" Guide

Goal
Turn your tacit skills into an explicit, reusable system.

Instructions

1. List the first three things you usually do when you get a new problem (be honest):
   - Step 1: 1st read and try it without thinking so error occurs
   - Step 2: Try to code the whole thing at once without planning.
   - Step 3: Ask a friend or give up
2. Now, design your ideal 6–8 step process (what a "future you" would always do):

1. Understand the real goal: Rewrite problem in my own words
2. Find inputs/outputs: List exactly what goes in and what must come out
3. Break it down: Split into 3-5 small, testable sub-problems on paper first
4. Design before coding
5. Build smallest testable piece: Code one function and test it
6. Add robustness.
7. (optional) Refactor for clarity
8. (optional) Document the "why"

3. For each step, write one question you will ask yourself:
   - Step 1 question: If I had to explain what would I say the program actually does
   - Step 2 question: What is the simplest example I can give.
   - Step 3 question: Can I test this piece alone in 5 lines of code
   - Step 4 question: Should this be a list, dict or class.
   - Step 5 question: Does this one function work before I write anything else
   - Step 6 question: What would break this? What error msg should the user see
4. Final instruction to yourself (one sentence):

"When I feel stuck, I will Stop coding, reread step 1 and solve the smallest piece I can test in 2 min